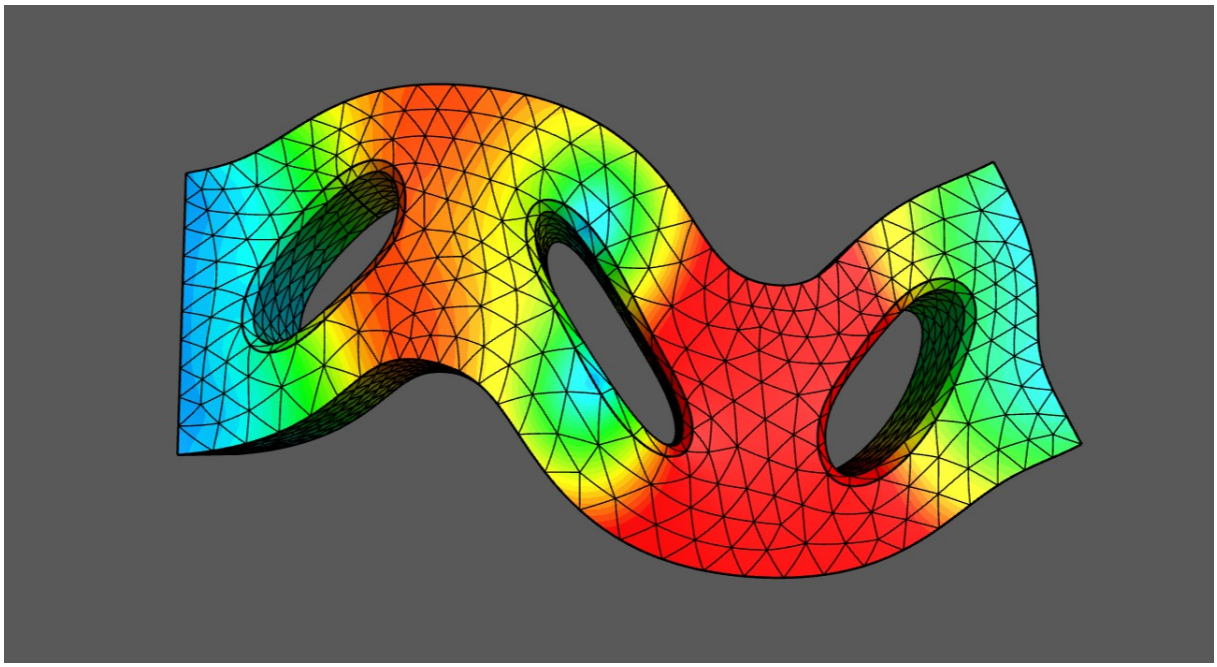
universität innsbruck

Elastodynamics Project

Hirtenlehner Max[*], Gröfler Tobias[†], Stein Christof[‡]

February 5, 2025



"Measuring programming progress by lines of code is like measuring aircraft building progress by weight."

Bill Gates

[*]max.hirtenlehner@student.uibk.ac.at
[†]tobias.groefler@student.uibk.ac.at
[‡]christof.stein@student.uibk.ac.at

Contents

1 Elastodynamics **1**

2 Analytical preparation **1**

2.1 Elastostatics 1

2.2 Undampened Elastodynamics 2

2.3 Dampened Elastodynamics 2

2.4 Newmark timestepping 3

2.5 Boundary conditions 4

3 Implementation **4**

3.1 Implementation without dampening 4

3.2 Implementation with dampening 5

3.3 Adding intricate Geometry 5

4 Results **6**

4.1 Results of Basic Geometry 6

4.2 Results of intricate Geometry 9

1 Elastodynamics

2 Analytical preparation

The theoretical part will follow a bottom up approach where we will start with the linear elastostatic equation. Then we will introduce more and more factors such as different boundary conditions and lastly time dependence and damping with the end result being a full elastodynamics model.

2.1 Elastostatics

First taking a look at the static equation for modelling material deformation we find

$$\nabla \boldsymbol{P}(\boldsymbol{u}) + \boldsymbol{F} = 0 \tag{1}$$

with $\boldsymbol{P}$ as the stress tensor of the model while $\boldsymbol{F}$ represents the internal forces. This is for the most part assumed to be 0 since we want to start from a relaxed state. $\boldsymbol{u}$ represents the deformation which we usually want to solve for. Since we will be mainly interested in linear elastodynamics, the tensor $\boldsymbol{P}$ reduces to the Cauchy stress tensor $\boldsymbol{\sigma}$:

$$\boldsymbol{\sigma} = 2\mu\boldsymbol{\varepsilon} + \lambda \operatorname{Tr}(\boldsymbol{\varepsilon})\, \boldsymbol{I} \tag{2}$$

with the stress tensor $\boldsymbol{\varepsilon}$ and lame parameters

$$\mu = \frac{E}{2(1+\nu)} \qquad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)} \tag{3}$$

The Strain is generally given by taking the gradient of deformation:

$$\boldsymbol{\varepsilon} = \nabla \boldsymbol{u} \tag{4}$$

Since we are only interested in the elastic deformation rather than the rigid body movement we can simply only take the symmetric part of the stress tensor. The entire Cauchy stress tensor for this model is therefore defined as

$$\boldsymbol{\sigma} = \frac{E}{(1+\nu)} \operatorname{Sym}(\nabla \boldsymbol{u}) + \frac{E\nu}{(1+\nu)(1-2\nu)} \operatorname{Tr}(\operatorname{Sym}(\nabla \boldsymbol{u}))\, \boldsymbol{I} \tag{5}$$

Luckily, deriving the variational formulation is comparatively simple for homogeneous Neumann conditions ($\nabla_{\boldsymbol{n}} \boldsymbol{u}\|_{\partial\Gamma} = \boldsymbol{f}$): By multiplying with a test function $\boldsymbol{v}$ and integrating yields :

$$\int_\Omega \boldsymbol{\sigma} : \operatorname{Sym}(\nabla \boldsymbol{v})\ \mathrm{dx} = \int_{\partial\Gamma} \boldsymbol{f}\boldsymbol{v}\ \mathrm{ds_x} - \int_\Omega \boldsymbol{F}\boldsymbol{v}\ \mathrm{dx} \tag{6}$$

which simplifies to

$$\underbrace{\int_\Omega \boldsymbol{\sigma} : \operatorname{Sym}(\nabla \boldsymbol{v})\ \mathrm{dx}}_{\boldsymbol{Au}} = \underbrace{\int_{\partial\Gamma} \boldsymbol{f}\boldsymbol{v}\ \mathrm{ds_x}}_{\boldsymbol{F}} \tag{7}$$

when assuming a relaxed system. This of course is a simple linear System

$$\boldsymbol{A}\boldsymbol{u} = \boldsymbol{F} \tag{8}$$

Which can be easily solved by inverting $\boldsymbol{A}$ in order to attain the deformation. Generally we will gather all force terms into $\boldsymbol{F}$.

2.2 Undampened Elastodynamics

If we want to investigate a time dependent solution of this equation we need to extend the original equation

$$\rho\frac{\partial^2 \boldsymbol{u}}{\partial t^2} + \nabla \boldsymbol{P}(\boldsymbol{u}) + \boldsymbol{F} = 0 \tag{9}$$

This introduces the additional second time derivative on the left hand side of the equation. This results in a model with large similarities to a driven physical oscillator with zero damping. The Variational formulation takes the form

$$\underbrace{\frac{\partial^2}{\partial t^2}\int_\Omega \boldsymbol{u} v \ \mathrm{dx}}_{\boldsymbol{M}\frac{\partial \boldsymbol{u}}{\partial t}} + \underbrace{\int_\Omega \boldsymbol{\sigma} : \mathrm{Sym}\left(\nabla \boldsymbol{v}\right) \ \mathrm{dx}}_{\boldsymbol{A}\boldsymbol{u}} = \underbrace{\int_{\partial\Gamma} \boldsymbol{f}\boldsymbol{v} \ \mathrm{ds_x} - \int_\Omega \boldsymbol{F}\boldsymbol{v} \ \mathrm{dx}}_{\boldsymbol{F}} \tag{10}$$

Interestingly we can analyse the non driven equation $\boldsymbol{F} = 0$ to find the eigenfrequencies and modes of the mesh.

$$\boldsymbol{M}\frac{\partial^2 \boldsymbol{u}}{\partial t^2} + \boldsymbol{A}\boldsymbol{u} = 0 \tag{11}$$

Where we first want to evaluate the eigenvalue problem defined by applying the Ansatz

$$\boldsymbol{u} = \boldsymbol{u_0} e^{i\omega t} \tag{12}$$

to the above equation

$$\boldsymbol{M}^{-1}\boldsymbol{A}\boldsymbol{u} = \omega^2 \boldsymbol{u} \tag{13}$$

and solving for the eigenvalues ω.

2.3 Dampened Elastodynamics

Lastly we want to introduce a damping term to the model to increase real world applications. Essentially we generate another (dampening) term $\boldsymbol{C}$ inside the initial partial differential equation:

$$\rho\frac{\partial^2 \boldsymbol{u}}{\partial t^2} - \frac{\partial \boldsymbol{c}}{\partial t} + \nabla \boldsymbol{P} + \boldsymbol{F} = 0 \tag{14}$$

using Maxwell damping assumes stress relaxation to model for example materials that are able to deform under long term stress, such as rubbers and other viscoelastic materials. This leaves the damping tensor $\boldsymbol{c}$ defined as

$$\frac{\partial \boldsymbol{c}}{\partial t} = \nabla\eta\frac{\partial \boldsymbol{\varepsilon}}{\partial t} = \frac{\partial}{\partial t}\nabla\eta\,\mathrm{Sym}\left(\boldsymbol{u}\right) \tag{15}$$

The resulting weak formulation can be written as

$$\underbrace{\frac{\partial^2}{\partial t^2}\int_\Omega \boldsymbol{u}\boldsymbol{v}\ \mathrm{dx}}_{\boldsymbol{M}\frac{\partial^2 \boldsymbol{u}}{\partial t^2}} - \underbrace{\frac{\partial}{\partial t}\int_\Omega \eta\boldsymbol{\varepsilon} : \mathrm{Sym}\left(\nabla v\right)\ \mathrm{dx}}_{\boldsymbol{C}\frac{\partial \boldsymbol{u}}{\partial t}} + \underbrace{\int_\Omega \boldsymbol{\sigma} : \mathrm{Sym}\left(\nabla \boldsymbol{v}\right)\ \mathrm{dx}}_{\boldsymbol{A}\boldsymbol{u}} = \underbrace{\int_{\partial\Gamma} \boldsymbol{f}\boldsymbol{v}\ \mathrm{ds_x} - \int_\Omega \boldsymbol{F}\boldsymbol{v}\ \mathrm{dx}}_{\boldsymbol{F}} \tag{16}$$

With the total linear system defined as

$$\boldsymbol{M}\frac{\partial^2 \boldsymbol{u}}{\partial t^2} + \boldsymbol{C}\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{A}\boldsymbol{u} = \boldsymbol{F} \tag{17}$$

2.4 Newmark timestepping

For Newmark time stepping we can now introduce additional grid functions $\boldsymbol{v}$ as the deformation velocity and $\boldsymbol{a}$ as the deformation acceleration:

$$\boldsymbol{v} = \frac{\partial \boldsymbol{u}}{\partial t} \qquad \boldsymbol{a} = \frac{\partial^2 \boldsymbol{u}}{\partial t^2} \tag{18}$$

as intermediate steps for the calculation since newmark stepping allows variation between fully implicit and fully explicit time stepping using the parameters β and γ. The corresponding general equations for deformation, velocity and acceleration are given by:

$$\boldsymbol{u}^{n+1} = \boldsymbol{u}^n + \Delta t \boldsymbol{v}^n + \frac{\Delta t^2}{2}\left[\left(1-2\beta\right)\boldsymbol{a}^n + 2\beta \boldsymbol{a}^{n+1}\right] \tag{19}$$

$$\boldsymbol{v}^{n+1} = \boldsymbol{v}^n + \left(1-\gamma\right)\Delta t \boldsymbol{a}^n + \gamma \Delta t \boldsymbol{a}^{n+1} \tag{20}$$

$$\boldsymbol{M}\boldsymbol{a}^{n+1} = \boldsymbol{F}^{n+1} - \boldsymbol{C}\boldsymbol{v}^{n+1} - \boldsymbol{A}\boldsymbol{u}^{n+1} \tag{21}$$

Since we get circular dependecies on the next time step information we have to resolve this using a linear system of equations. First we combine all three equations:

$$\begin{aligned}\boldsymbol{M}\boldsymbol{a}^{n+1} =& \boldsymbol{F}^{n+1} \\ &- \boldsymbol{C}\left[\boldsymbol{v}^n + \left(1-\gamma\right)\Delta t \boldsymbol{a}^n + \gamma \Delta t \boldsymbol{a}^{n+1}\right] \\ &- \boldsymbol{A}\left[\boldsymbol{u}^n + \Delta t \boldsymbol{v}^n + \frac{\Delta t^2}{2}\left(\left(1-2\beta\right)\boldsymbol{a}^n + 2\beta \boldsymbol{a}^{n+1}\right)\right]\end{aligned} \tag{22}$$

Where we can isolate the new acceleration time step to the left hand side:

$$\underbrace{\left(\boldsymbol{M} - \gamma \boldsymbol{C}\Delta t + 2\beta \boldsymbol{A}\right)}_{\boldsymbol{M}^*}\boldsymbol{a}^{n+1} = \boldsymbol{F}^{n+1} \quad + \boldsymbol{C}\left[\boldsymbol{v}^n + \left(1-\gamma\right)\Delta t \boldsymbol{a}^n\right] - \boldsymbol{A}\left[\boldsymbol{u}^n + \Delta t \boldsymbol{v}^n + \frac{\Delta t^2}{2}\left(1-2\beta\right)\boldsymbol{a}^n\right] \tag{23}$$

This allows us to solve the system by inverting $\boldsymbol{M}^*$ for the new time step and generate our time iteration. The resulting velocities and displacements can then easily be calculated using equations 20 and 19 respectively.

2.5 Boundary conditions

We will be using only Dirichlet and Neumann Boundary conditions. Since inhomogeneous Dirichlet boundaries represent deformation directly we would like to use these since they are the most predictable for inducing specific deformations. Additionally using homogeneous Dirichlet boundaries equates to affixing certain mesh boundaries to one space. Inhomogeneous Dirichlet boundaries do however introduce additional terms into the already quite extensive formulation and will therefore be shown here. By splitting the deformation into homogeneous $\boldsymbol{u}_0$ and inhomogeneous $\boldsymbol{u}_g$ parts we gain the solution for the homogeneous part like seen in equation 16 with the addition of the right hand side term

$$\boldsymbol{D} = \int_\Omega \rho \frac{\partial^2 \boldsymbol{u}_g}{\partial t^2} \cdot \boldsymbol{v}\, dx + \int_\Omega \boldsymbol{C} \frac{\partial \boldsymbol{u}_g}{\partial t} \cdot \boldsymbol{v}\, dx - \int_\Omega \boldsymbol{\sigma}(\boldsymbol{u}_g) : \mathrm{Sym}\left(\nabla \boldsymbol{v}\right)\, dx. \tag{24}$$

The solution has to be recovered using

$$\boldsymbol{u} = \boldsymbol{u}_0 + \boldsymbol{u}_g \tag{25}$$

Neumann Boundary conditions are however trivial to implement in this since ng solve offers this functionality directly when calculating the force term $\boldsymbol{F}$

3 Implementation

For the implementation we first use a geometry that is easy to show. For this purpose we use a Box with three cylinder cutouts. The boundaries are given as the outer borders of the box and another boundary is are the cylinders inside. In the final step we try it with an intricate Geometry. There will be two implementations one with dampening one without.

3.1 Implementation without dampening

In a first step certain coefficients will have to be initialized. First and foremost the time-step as well as the beta and gamma used in the Newmark-beta method [1]. Where beta is between 0 and 1 and so is gamma. But to get an unconditionally stable scheme the limit for beta is 0.25. If it is set to higher values and this there is more dampening. Gamma is set to 0.5 to make it a middle point rule. Then E and ν need to be defined to get the parameters shown in equation 3. With these coefficients the stress function can be defined as a function of strain. Then the Grid Functions need to be defined we work with and order= 3 mesh. Three Grid functions are defined. With the TaskManager the definitions of the Billinear Forms is done. First the Mass matrix is initialised with $\boldsymbol{u} \cdot \boldsymbol{v} \cdot dx$ and then assembled. Then the Billinear Form of the Stress Matrix is made up of the inner product of the Stress(Sym($\nabla \cdot \boldsymbol{u}$)) and ($\nabla \cdot \boldsymbol{v}$). And is then Assembled as well. To do the calculations we also need the inverse in the form of $\boldsymbol{M}^*$ which is

$$\boldsymbol{u} \cdot \boldsymbol{v} \cdot dx + \beta \Delta t^2 \cdot \mathrm{Stress}(\mathrm{Sym}(\nabla \cdot \boldsymbol{u}) : (\nabla \cdot \boldsymbol{v}) dx. \tag{26}$$

Before the Assembly of $\boldsymbol{M}^*$ we define a *MultiGridPreconditioner* on $\boldsymbol{M}^*$. For the right hand side we define the force. And then the Linear-form f as $\boldsymbol{F} \cdot \boldsymbol{v} \cdot ds$. In the next step the

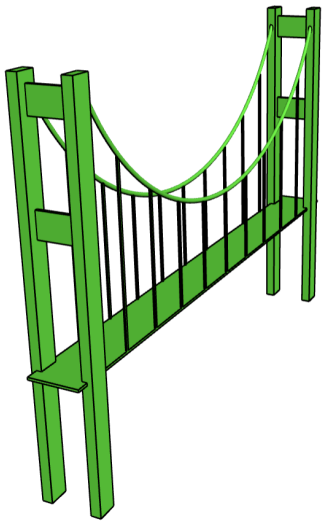
CGSolver from the *ngsolve.krylovspace* is used to solve the linear system from $\boldsymbol{M}^*$.mat. This is performance enhanced by the before defined preconditioner. Now we come to the Newmark time stepping which is explained in detail in the first chapter. To use it in ngsolve we need to create gridfunctions for the first time derivative and the second and set them Coefficient functions as can be seen in the code. We then need a scalar Grid function on the Grid function space. Then a Matrix Valued finite element space is created with order three and a Grid function as well as a scalar Grid function on that space. After defining the end time as well as the steps the Newmark Time stepping begins. As this is outlined in the chapter on Newmark time stepping here we won't go into greater detail. The only difference is that the Term with the C is falling away since we are in the part without dampening. In the Time stepping we have to add the Displacement Data to the Multi-Dimensional Grid Function then the stress field is interpolated to the gridfunction. The Stress Grid function is then flattened. After the Time stepping the solutions are Drawn with the Draw function.

3.2 Implementation with dampening

The Implementation with dampening is essentially the same only that we add a term into the $\boldsymbol{M}^*$ as well as an additional Billinear Form C. For this a new constant needs to be added, η. This is used to create C. It is initialised as the Inner Product of $\eta\text{Sym}(\nabla \boldsymbol{u})$) and $\text{Sym}(\nabla \boldsymbol{v})$ times dx. this term also goes into $\boldsymbol{M}^*$.

3.3 Adding intricate Geometry

Motivated by the well-known example of the collapse of the Tacoma Bridge 1b, we then added an more intricate geometry in the form of a bridge shown in figure 1a.



(a) Constructed Bridge Geometry with netgen.occ.



(b) Collapse of Tacoma Bridge due to resonance that occurred by winds. Source: `https://www.britannica.com/topic/Tacoma-Narrows-Bridge`

Here, the geometry was constructed using the netgen.occ library. Due to limitations of this library and also in computational power, we only made a simplified model of the bridge. The bridges pillar are fixed in place by the BC. This also reduces computational costs. On the upper

surface of the road, the force is applied. Due to computational limits, the geometry needed to be simplified a little bit more for the time stepping. Otherwise the simulation took too much time to animate.

In a next step we import the Stanford Bunny to do a simulation on it. For that we downloaded the Stanford bunny geometry and reduced the size and amount of triangles. It is then imported as an STL file with the STLGeometry Package from Netgen. Then the boundaries are set via manual adjustment inside the netgen mesh objects. In figure 2 the meshed Rabbit is shown with highlighted boundaries.

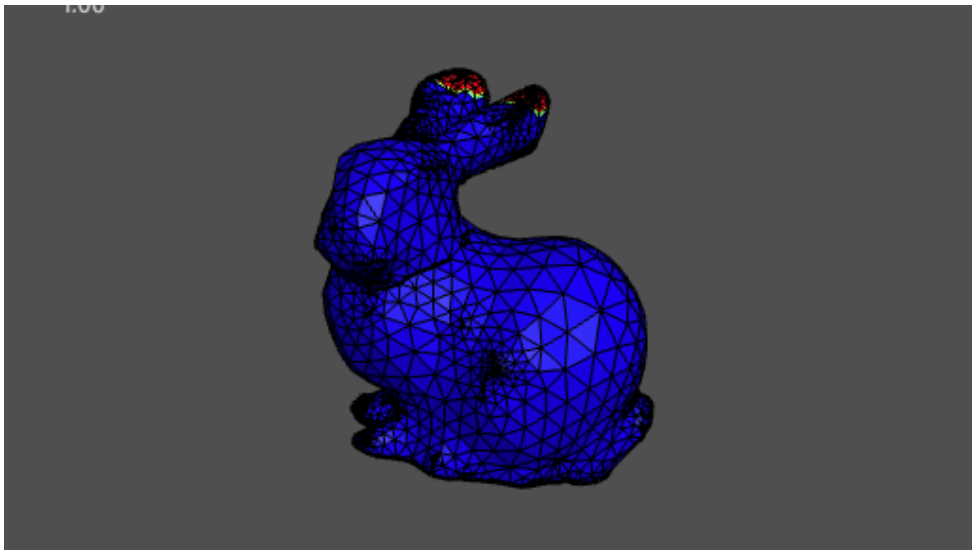


Figure 2: Rabbit Geometry with highlighted upper boundaries

4 Results

Here a few results are shown and discussed.

4.1 Results of Basic Geometry

We start with a basic Geometry namely the brick with cylindrical cutouts. The Animation can be found in the gitlab the figures are only snippets. In figures 3, 4 and 5 the working simulation is shown out of two different perspectives and timesteps. In figures 6 and 7 we see snippets of a simulation in which the applied force deforms the Geometry completely. Since the applied force is oscillating it could be that we hit the resonance frequency and have an instant of a resonance catastrophe or something is going wrong in the timestepping but since it works for all other cases we suspect the first.

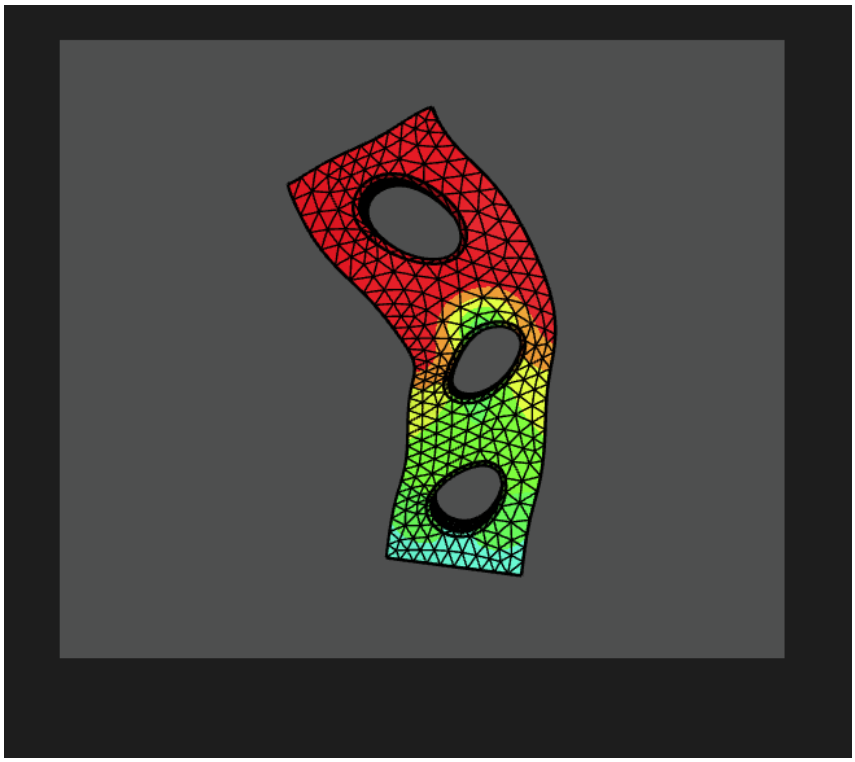


Figure 3: Working Brick Simulation

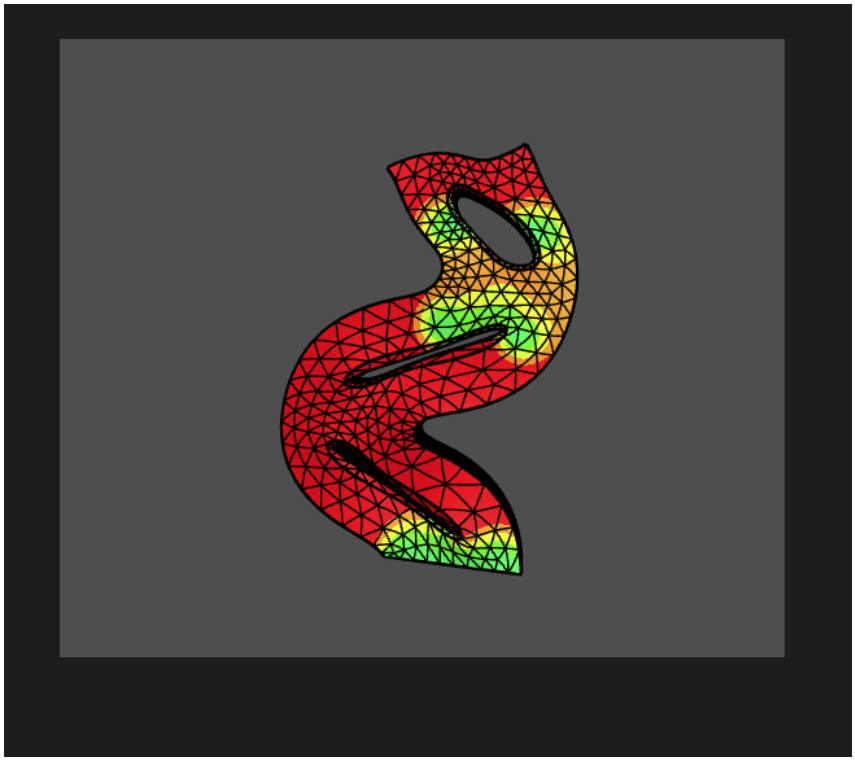


Figure 4: Working Brick Simulation

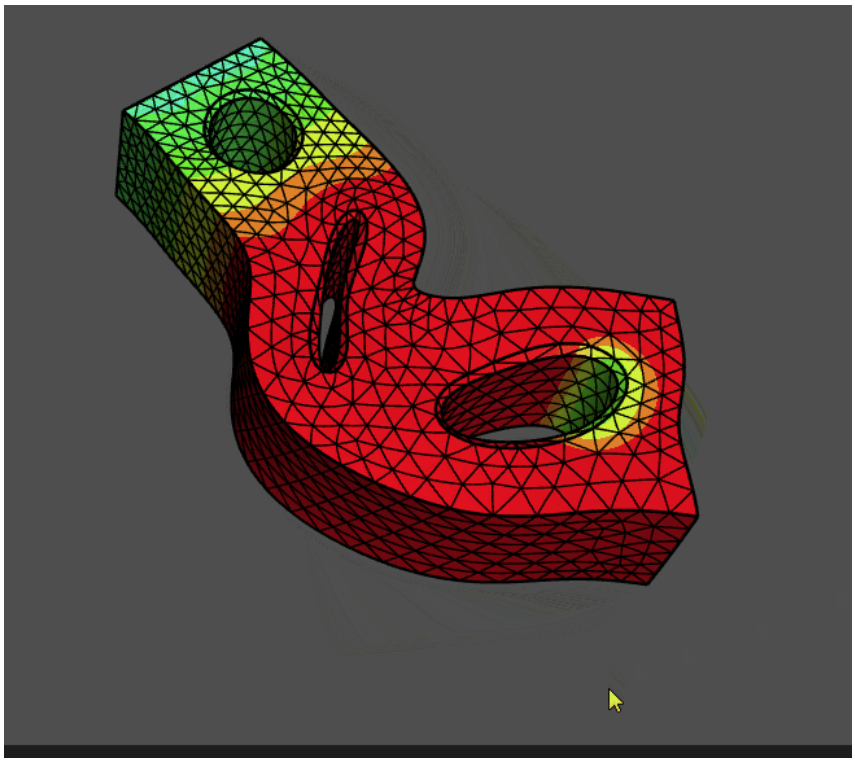


Figure 5: Working Brick Simulation

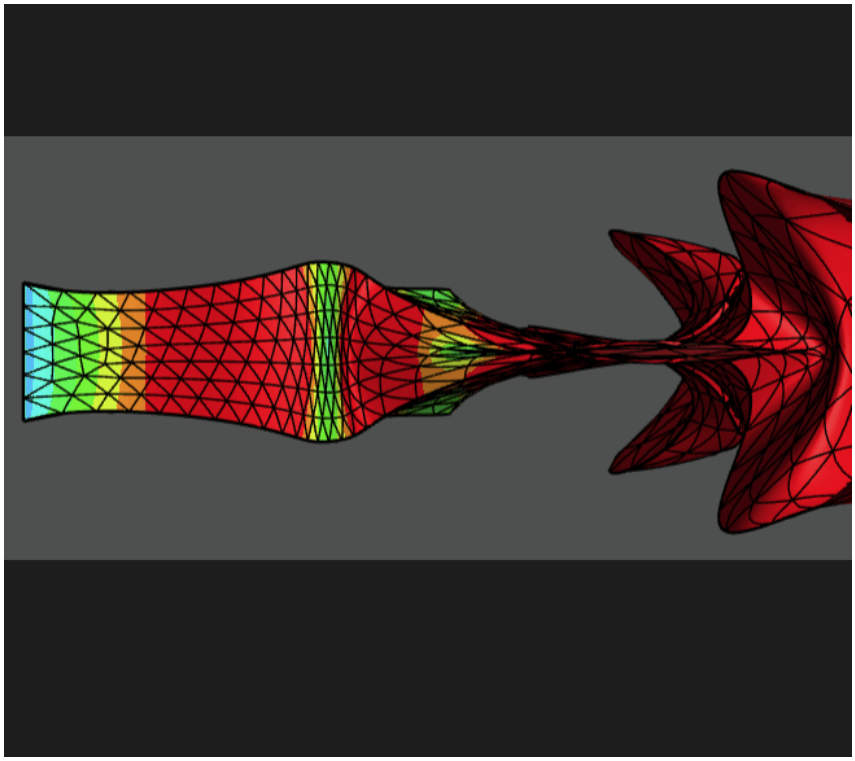


Figure 6: Failing Brick Simulation

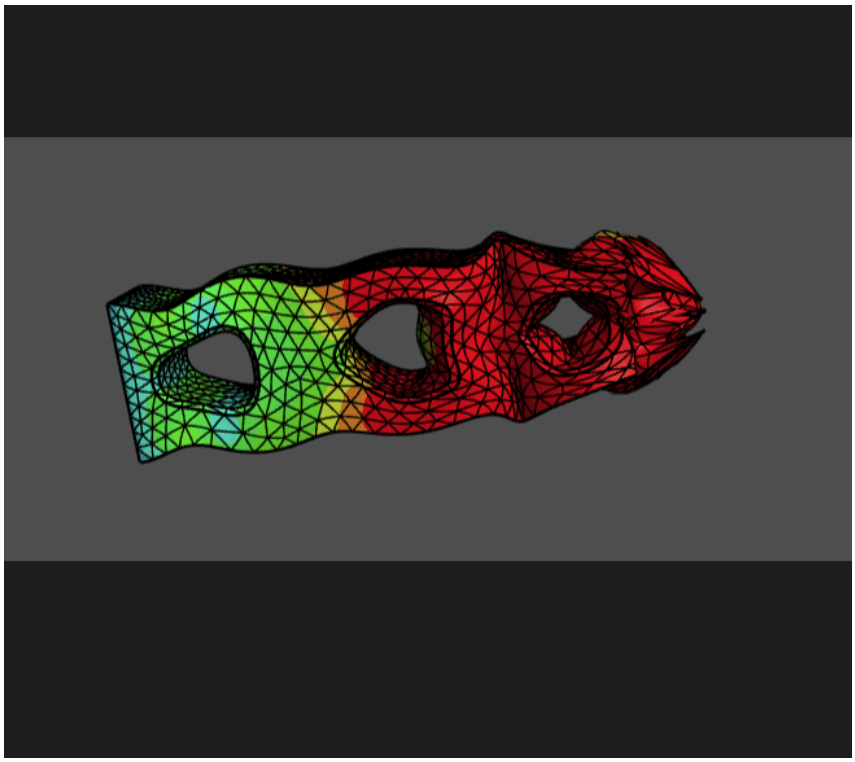


Figure 7: Failing Brick Simulation

4.2 Results of intricate Geometry

Now we look at the Bridge geometry from figure 1a. Like mentioned, we needed to simplify the geometry a little bit more to prevent the code from running too long. So not all support ropes from the bridge geometry got used. The resulting geometry from the elastodynamics without dampening is shown in figure 8.

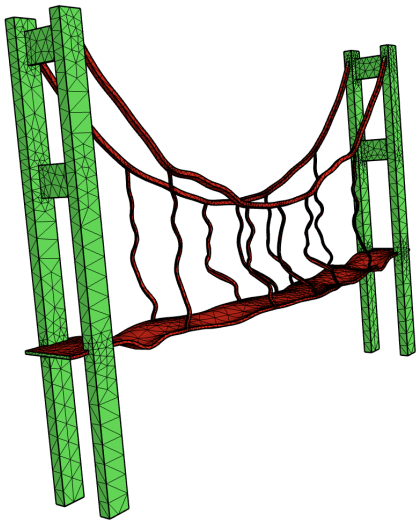


Figure 8: Simple bridge geometry after some time steps without dampening.

We notice, that the resulting geometry from the simulation is not providing a physical solution. This is the case due to the lack of gravity in the simulation as well as wrong material properties.

Also the force, which should represent a wind that bring the bridge to swing, is not implemented realistically enough. In the original Tacoma bridge, the walls on the side of the bridge caused the sideways wind to exert an force on it. This caused the bridge to swing and that with its resonance frequency. That is also not considered in our example or is not implemented in a simplified way.

The Animation of the Rabbit can be found also in the repository. A snipped of this can be seen in figure 9. Here we try to tug at its ear.

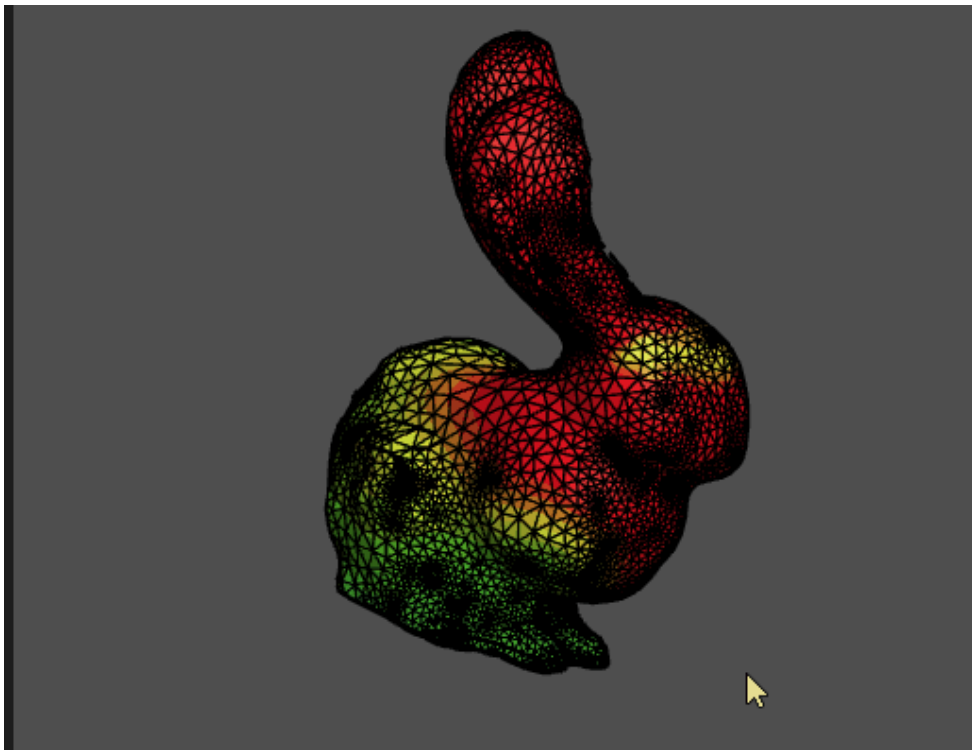


Figure 9: Stanford Rabbit Simulation

References

[1] N. M. Newmark. **?**A method of computation for structural dynamics**? in***Journal of the engineering mechanics division*: 85.3 (1959), **pages** 67–94.